Agentic Inbox Processing

An intelligent document classification system for Private Equity (PE) lifecycle events using Azure AI Agent Service. Ingests documents from **email** (via Logic App + Microsoft Graph) and **SFTP** (via Logic App + SFTP connector), classifies them into 11 PE categories with multi-language support (English/French), and routes them based on classification confidence.

Table of Contents

- Overview
- Architecture
- Prerequisites
- Azure Services Setup
- Local Installation
- Configuration
- Running the Application
- Testing
- Project Structure
- PE Event Categories
- Pipeline Configuration
- Authentication (Entra ID Easy Auth)
- Deploying to Azure
- Connections and Secrets Management
- Troubleshooting

Overview

This solution provides:

- **Dual Intake**: Ingests documents from email (Microsoft Graph) and SFTP (polling trigger)
- **Automated Classification**: Uses Azure AI Agent Service (GPT-4o) to classify incoming PE documents
- **Multi-step Processing**: 2-stage classification (relevance check → detailed categorization)
- **Attachment Analysis**: Extracts text from PDF attachments using Azure Document Intelligence
- **Download-Link Intake**: Detects download links in email bodies, downloads the documents, and stores them in Azure Blob Storage
- **SFTP File Ingestion**: Polls SFTP `/in/` folder, backs up to blob storage, routes spreadsheets to SharePoint and PDFs to classification agent
- **Content Hash Deduplication**: Detects duplicate uploads and content updates using blob Content-MD5 hashes with 3-way routing (new / duplicate / update)
- **Entity Extraction**: Automatically extracts fund names, PE company names, amounts, and dates
- **Confidence-based Routing**: Routes emails to different queues based on classification confidence
- **Pipeline Configuration**: Switch between full classification pipeline and triage-only mode via environment variable
- **Configurable Queue Names**: All queue names are configurable via environment variables
- **Web Dashboard**: Real-time monitoring of processing status, queue contents, pipeline mode indicator, delivery tracking for SFTP records
- **Deduplication**: Prevents duplicate PE events using intelligent hashing

Architecture

```
                    ┌── INTAKE SOURCES ──┐
                    │                    │
  ┌─────────────────┼                    ├──────────────────────┐
  │                 │                    │                      │
  ▼                 │                    │                      ▼
┌─────────────────┐ │                    │   ┌──────────────────────────┐
│ Email Source    │ │                    │   │ SFTP Server (/in/)       │
│ (Outlook/API)   │ │                    │   └──────────────────────────┘
└─────────────────┘ │                    │                 │
        │           │                    │                 ▼
        ▼           │                    │   ┌──────────────────────────┐
┌─────────────────┐ │                    │   │ Logic App (SFTP Ingestion)│
│ Logic App       │ │                    │   │                          │
│ (Email)         │ │                    │   │ 1. Download file content │
│                 │ │                    │   │ 2. Upload to Blob Storage│
│ Graph API       │ │                    │   │ 3. Get Content-MD5 hash  │
│ ingestion       │ │                    │   │ 4. 3-way dedup check     │
└─────────────────┘ │                    │   │ 5. Route by file type    │
        │           │                    │   │ 6. Delete from SFTP      │
        │           │                    │   └──────────────────────────┘
        │           │                    │         │         │
        ▼           │                    │         │         │
┌─────────────────┐ │                    │      CSV/XLS     PDF
│ Service Bus     │◀┘                    │         │         │
│ intake          │                      └─────────┼─────────┘
└─────────────────┘                                │
        │                                          ▼
        ▼                                 ┌────────────────────┐
┌──────────────────────────────────┐      │ SharePoint         │
│ Email Classification Agent       │      │ (Graph API PUT)    │
│ (Azure AI Agent Service)         │      └────────────────────┘
│                                  │
│ Step 1: Relevance Check          │
│ Step 2: Route by Pipeline Mode   │
└──────────────────────────────────┘
                │
    ┌───────────┴──────────┐
    │                      │
PIPELINE_MODE        PIPELINE_MODE
  = "full"            = "triage-only"
    │                      │
    ▼                      ▼
┌──────────────┐   ┌────────────────────┐
│ Classify     │   │ triage-complete    │
│ Extract      │   │ (for IDP /         │
└──────────────┘   │  downstream)       │
        │          └────────────────────┘
        │
  ┌─────┼──────────────┐
  │     │              │
  ▼     ▼              ▼
┌────┐┌──────────────┐┌──────────────┐
│disc││archival-     │human-        │
│ard-│pending       │review        │
│ed  │(≥65% conf.)  │(<65% conf.)  │
└────┘└──────────────┘└──────────────┘
```

```
                │
                ▼
        ┌──────────────────────────┐
        │ Cosmos DB                │
        │ (intake-records)         │
        │ PK: /partitionKey        │
        └──────────────────────────┘
```

Prerequisites

Required Software

- **Python 3.12+** - Download Python
- **Azure CLI** - Install Azure CLI
- **Git** - Install Git

Azure Subscription

You need an active Azure subscription with permissions to create the required resources.

Azure Services Setup

Create the following Azure services (or use the provided Bicep templates in `/infrastructure`):

1. Azure AI Foundry (AI Agent Service)

- Create an Azure AI Foundry resource
- Deploy a GPT-4o model
- Note the endpoint URL and project name

2. Azure Service Bus

- Create a Service Bus namespace (Standard tier or higher)
- Create the following queues:
  - `intake` - Incoming emails (from email Logic App + SFTP Logic App)
  - `discarded` - Non-PE emails
  - `human-review` - Low confidence classifications
  - `archival-pending` - Successfully classified emails
  - `triage-complete` - Triage-only mode output (for IDP / downstream systems)

3. Azure Cosmos DB

- Create a Cosmos DB account (NoSQL API)
- Create a database named `email-processing`
- Create containers:
  - `intake-records` (partition key: `/partitionKey`)
    - Email records: partition key = `{sender_domain}_{YYYY-MM}`
    - SFTP records: partition key = `{sftp_username}_{YYYY-MM}`

4. Azure Document Intelligence (Optional)

- Create a Document Intelligence resource
- Note the endpoint URL (for PDF attachment processing)

5. Azure Storage Account

- Create a Storage Account (for blob backup of ingested documents)
- Create a blob container named `attachments`
- Note the storage account name
- Ensure your identity (and the SFTP Logic App managed identity) has `Storage Blob Data Owner` role

6. Role Assignments

Ensure your Azure identity has the following roles:

- **Service Bus**: `Azure Service Bus Data Sender` and `Azure Service Bus Data Receiver`
- **Cosmos DB**: `Cosmos DB Built-in Data Contributor`
- **Document Intelligence**: `Cognitive Services User`
- **AI Foundry**: `Azure AI Developer`
- **Storage Account**: `Storage Blob Data Contributor`

Infrastructure as Code (Optional)

Deploy all resources using Bicep:

```
cd infrastructure
az deployment group create \
  --resource-group <your-rg> \
  --template-file main.bicep \
  --parameters parameters/dev.bicepparam
```

Local Installation

Step 1: Clone the Repository

```
git clone <repository-url>
cd agentic-inbox-processing
```

Step 2: Create Virtual Environment

Windows (PowerShell)

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Linux/macOS

```
python3 -m venv .venv
source .venv/bin/activate
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Azure Authentication

```
# Login to Azure
az login

# Set your subscription (if you have multiple)
az account set --subscription "<subscription-name-or-id>"

# Verify authentication
az account show
```

Configuration

Step 1: Create Environment File

Create a `.env` file in the project root with the following variables:

```
# Azure AI Agent Service
AZURE_AI_PROJECT_ENDPOINT=https://<your-ai-
foundry>.services.ai.azure.com/api/projects/<project-id>
AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4o

# Azure Service Bus
SERVICEBUS_NAMESPACE=<your-servicebus-namespace>

# Azure Cosmos DB
COSMOS_ENDPOINT=https://<your-cosmos-account>.documents.azure.com:443/
COSMOS_DATABASE=email-processing

# Azure Document Intelligence (optional - for PDF attachments)
DOCUMENT_INTELLIGENCE_ENDPOINT=https://<your-doc-intel>.cognitiveservices.azure.com/

# Azure Storage Account (for blob backup of ingested documents)
STORAGE_ACCOUNT_NAME=<your-storage-account-name>

# Azure Key Vault (stores Graph and SharePoint client secrets)
KEY_VAULT_URL=https://<your-key-vault>.vault.azure.net/

# Microsoft Graph API (client secret is fetched from Key Vault at runtime)
# GRAPH_CLIENT_ID=<app-registration-client-id>
# GRAPH_TENANT_ID=<your-tenant-id>

# Pipeline Configuration
PIPELINE_MODE=full                        # "full" (default) or "triage-only"
TRIAGE_COMPLETE_QUEUE=triage-complete      # Queue name for triage-only output
# TRIAGE_COMPLETE_SB_NAMESPACE=<external>  # Optional: external Service Bus namespace

# Queue Names (override defaults if needed)
HUMAN_REVIEW_QUEUE=human-review
```

```
ARCHIVAL_PENDING_QUEUE=archival-pending
DISCARDED_QUEUE=discarded
```

Step 2: Validate Configuration

```
python utils/diagnose.py
```

This script checks:

- Environment variables are set
- Azure authentication is working
- Service Bus connectivity
- Cosmos DB connectivity

Running the Application

Option 1: Web Dashboard

Start the FastAPI dashboard to monitor email processing:

Windows

```
.\.venv\Scripts\python.exe -m uvicorn src.webapp.main:app --reload --port 8000
```

Linux/macOS

```
python -m uvicorn src.webapp.main:app --reload --port 8000
```

Open your browser to: http://127.0.0.1:8000

The dashboard shows:

- Active pipeline mode indicator (Full Pipeline / Triage Only badge)
- Emails in each queue (intake, discarded, human-review, archival-pending, triage-complete)
- Processed emails from Cosmos DB with per-email pipeline status
- Classification results and confidence scores

Option 2: Email Classification Agent

Process a single email (best for testing):

```
# Windows
.\.venv\Scripts\python.exe src/agents/run_agent.py --once

# Linux/macOS
python src/agents/run_agent.py --once
```

Continuous Processing (polls queue every 30 seconds):

```
# Windows
.\.venv\Scripts\python.exe src/agents/run_agent.py

# Linux/macOS
python src/agents/run_agent.py
```

Custom Settings:

```
python src/agents/run_agent.py --max-emails 50 --wait-seconds 60
```

Option 3: Run Both Together

Open two terminal windows:

Terminal 1 - Dashboard:

```
.\.venv\Scripts\Activate.ps1
python -m uvicorn src.webapp.main:app --reload --port 8000
```

Terminal 2 - Agent:

```
.\.venv\Scripts\Activate.ps1
python src/agents/run_agent.py
```

Testing

Send a Test Email to the Queue

```
from azure.identity import DefaultAzureCredential
from azure.servicebus import ServiceBusClient, ServiceBusMessage
import json, os
from dotenv import load_dotenv

load_dotenv('.env')
namespace = os.environ['SERVICEBUS_NAMESPACE']

cred = DefaultAzureCredential()
client = ServiceBusClient(f'{namespace}.servicebus.windows.net', credential=cred)
sender = client.get_queue_sender('intake')

email = {
    'id': 'test-001',
    'subject': 'Capital Call Notice - Q1 2025',
    'from': 'fund@example.com',
    'receivedAt': '2025-01-15T10:30:00Z',
```

```
    'bodyText': 'Capital Call Amount: EUR 2,500,000. Due Date: January 30, 2025. Fund:
Private Equity Fund XV.',
    'hasAttachments': False,
    'attachments': []
}

sender.send_messages(ServiceBusMessage(json.dumps(email)))
print('Test email sent to intake queue!')
sender.close()
client.close()
```

Check Queue Status

```
python src/peek_queue.py
```

Run Connectivity Tests

```
python utils/test_connectivity.py
```

Run Automated Tests

```
# Run all tests
pytest

# Run unit tests only
pytest tests/unit/

# Run integration tests only
pytest tests/integration/

# Run with verbose output
pytest -v
```

Project Structure

```
agentic-inbox-processing/
├── src/
│   ├── agents/                    # Email classification agents
│   │   ├── email_classifier_agent.py  # Main classification logic
│   │   ├── classification_prompts.py  # LLM prompts and schemas
│   │   ├── run_agent.py               # CLI runner script
│   │   └── tools/                     # Agent tools
│   │       ├── cosmos_tools.py        # Cosmos DB operations
│   │       ├── document_intelligence_tool.py  # PDF extraction
│   │       ├── graph_tools.py         # Microsoft Graph API
│   │       ├── link_download_tool.py  # Download-link detection & blob upload
│   │       └── queue_tools.py         # Service Bus operations
│   └── webapp/                    # FastAPI web dashboard
```

```
│       ├── main.py                    # Dashboard application
│       └── templates/                 # HTML templates
│           └── dashboard.html
├── infrastructure/                # Azure Infrastructure as Code
│   ├── main.bicep                    # Main deployment template
│   ├── modules/                      # Bicep modules
│   └── parameters/                   # Environment parameters
├── logic-apps/                    # Logic App workflows
│   ├── email-ingestion/               # Email intake (Graph API trigger)
│   └── sftp-file-ingestion/           # SFTP intake (polling trigger)
├── tests/                         # Automated tests
│   ├── unit/                          # Unit tests
│   │   ├── test_link_download_tool.py     # Link download tool tests
│   │   └── test_pipeline_config.py        # Pipeline mode tests
│   └── integration/                   # Integration tests
│       ├── test_link_download_flow.py     # End-to-end link download flow
│       └── test_sftp_intake_flow.py       # SFTP intake integration tests
├── specs/                         # Feature specifications
│   ├── 001-download-link-intake/      # Download-link intake spec
│   ├── 002-pipeline-config/           # Pipeline configuration spec
│   └── 003-sftp-intake/               # SFTP file ingestion spec
├── utils/                         # Utility scripts
│   ├── diagnose.py                   # Configuration checker
│   ├── test_connectivity.py          # Connection tests
│   ├── purge_queues.py               # Queue maintenance
│   ├── clear_cosmos_emails.py        # Data cleanup
│   └── migrate_container.py          # Cosmos DB partition key migration
├── requirements.txt               # Python dependencies
├── pyproject.toml                 # Project metadata & pytest config
├── startup.sh                     # Azure App Service startup
├── gunicorn.conf.py              # Production server config
└── README.md                      # This file
```

PE Event Categories

The system classifies emails into these Private Equity lifecycle events:

Category	Description
Capital Call	Request for committed capital from investors
Distribution Notice	Distribution of proceeds to investors
Capital Account Statement	Periodic account balance statement
Quarterly Report	Quarterly fund performance report
Annual Financial Statement	Year-end financial statements
Tax Statement	K-1 or tax-related documents
Legal Notice	Legal communications and notices
Subscription Agreement	New subscription documents
Extension Notice	Fund term extension notices
Dissolution Notice	Fund wind-down notifications

Category	Description
Not PE Related	Non-PE email (routed to discarded)

Service Bus Queues

Queue	Env Variable	Default	Routing Condition
intake	`INTAKE_QUEUE`	`intake`	Entry point (emails + SFTP PDFs)
discarded	`DISCARDED_QUEUE`	`discarded`	Not PE Related classification
archival-pending	`ARCHIVAL_PENDING_QUEUE`	`archival-pending`	Confidence ≥ 65%
human-review	`HUMAN_REVIEW_QUEUE`	`human-review`	Confidence < 65%
triage-complete	`TRIAGE_COMPLETE_QUEUE`	`triage-complete`	`PIPELINE_MODE=triage-only`

PE Event Deduplication

PE events are deduplicated to prevent the same event from being created multiple times when duplicate emails arrive. A **deduplication key** (SHA256 hash, first 16 chars) is generated from these normalized fields:

Field	Description	Normalization
`pe_company`	PE firm name	Lowercase, trimmed, common suffixes removed (llc, lp, inc, corp, ltd, partners, fund)
`fund_name`	Fund name	Same normalization as pe_company
`event_type`	Type of event (Capital Call, Distribution, etc.)	Lowercase, trimmed
`amount`	Transaction amount (optional)	Only digits and decimal point kept
`due_date`	Due date (optional)	Extracted to `YYYY-MM` format (month precision)

Download-Link Intake

The system automatically detects download links in email bodies and downloads the referenced documents. This handles the common scenario where PE firms send emails containing links to documents hosted on portals or cloud storage instead of traditional attachments.

How It Works

1. **URL Extraction** — Parses both plain-text and HTML email bodies for HTTP/HTTPS URLs
2. **Document Filtering** — Only attempts downloads for URLs that appear to reference documents (`.pdf`, `.docx`, `.xlsx`, `.csv`, `.pptx`, `.txt`, `.zip`); skips social media and non-document domains
3. **Download & Upload** — Downloads the document via HTTPS (with a 50 MB size limit) and uploads it to Azure Blob Storage at `attachments/{emailId}/{filename}`
4. **Record Enrichment** — Updates the Cosmos DB email record with the attachment path and sets `hasAttachments` to `true`
5. **Graceful Failures** — If a download fails (timeout, 404, auth-required, non-document content), the email is still processed normally; failures are logged for operational visibility

Dashboard Indicators

The web dashboard shows a link icon next to attachments that were sourced from download links (vs. traditional email attachments), making it easy to identify the origin of each document.

SFTP File Ingestion

The system ingests documents from an SFTP server via a Logic App that polls the `/in/` folder. Files are backed up to Azure Blob Storage, deduplicated using content hashes, and routed based on file type.

Workflow Steps

Step	Action	Description
1	Get file content	Downloads the file from the SFTP server
2	Generate file ID	Creates a unique `sftp-{guid}` identifier
3	Parse filename	Extracts file extension and metadata parts (Account, Fund, DocType, etc.)
4	Upload to blob	Backs up file to `/attachments/{fileId}/{filename}`
5	Get blob MD5	HTTP HEAD to Blob REST API for `Content-MD5` hash
6	Compute dedup key	`base64(sftpPath)` for O(1) Cosmos DB point-reads
7	3-way dedup check	New file → create record; Same hash → duplicate; Different hash → update
8	Route by file type	CSV/XLS/XLSX → SharePoint; PDF → Service Bus for classification
9	Delete from SFTP	Removes the file from `/in/` using file ID (not path, to avoid UTF-8 issues)

Content Hash Deduplication

The SFTP workflow uses blob `Content-MD5` for 3-way dedup routing:

Scenario	Cosmos DB Lookup	Content Hash Match	Action
New file	404 (not found)	N/A	Create intake record, route to queue
Duplicate	Found	Same as stored	Increment `deliveryCount`, append to `deliveryHistory`, terminate
Update	Found	Different	Increment `version` + `deliveryCount`, update `contentHash` and `blobPath`

Delivery Tracking Fields

Each SFTP intake record in Cosmos DB includes:

Field	Description	Example
`contentHash`	Blob Content-MD5 (base64)	`Lyaf8xLRAAIvloNxXOuaOQ==`
`version`	Content version (increments on update)	`2`
`deliveryCount`	Total deliveries (new + duplicate + update)	`3`
`deliveryHistory`	Array of delivery events	`[{deliveredAt, contentHash, action}]`

Field	Description	Example
`lastDeliveredAt`	Timestamp of most recent delivery	`2026-03-17T15:31:22Z`

File Type Routing

File Type	Destination	Method
CSV, XLS, XLSX	SharePoint document library	Graph API PUT (organized by `/{Letter}/{Account}/{Fund}/`)
PDF	Service Bus `intake` queue	Classification agent processes it
Other	Logged and skipped	File remains in blob storage

SFTP Filename Convention

Files should follow the delimiter-separated naming convention (default delimiter: `_`):

```
{Account}_{Fund}_{DocType}_{DocName}_{PublishedDate}_{EffectiveDate}.{ext}
```

Example: `AcmeCorp_FundXV_CapitalCall_Q1Notice_2026-01-15_2026-01-30.pdf`

Pipeline Configuration

The agent supports two pipeline modes, controlled by the `PIPELINE_MODE` environment variable:

Mode	Value	Behavior
Full Pipeline	`full` (default)	Relevance check → Classification → Entity extraction → Confidence-based routing
Triage Only	`triage-only`	Relevance check only → Forward to `triage-complete` queue for downstream processing

When to Use Each Mode

- **Full Pipeline (`full`)** — The agent handles end-to-end classification and routing. Emails are classified into PE event categories, entities are extracted, and the email is routed to `archival-pending`, `human-review`, or `discarded` based on confidence.
- **Triage Only (`triage-only`)** — The agent performs the initial relevance check (PE-related vs. not) and stops. PE-relevant emails are placed on the `triage-complete` queue for consumption by an external system (e.g., an IDP platform). Non-PE emails are still routed to `discarded`. Classification and entity extraction are skipped.

Environment Variables

Variable	Required	Default	Description
`PIPELINE_MODE`	No	`full`	`full` or `triage-only`
`TRIAGE_COMPLETE_QUEUE`	No	`triage-complete`	Queue name for triage-only output
`TRIAGE_COMPLETE_SB_NAMESPACE`	No	*(unset)*	Optional external Service Bus namespace. If unset, the primary namespace is used.

Variable	Required	Default	Description
`HUMAN_REVIEW_QUEUE`	No	`human-review`	Queue name for low-confidence classifications
`ARCHIVAL_PENDING_QUEUE`	No	`archival-pending`	Queue name for classified emails (≥65%)
`DISCARDED_QUEUE`	No	`discarded`	Queue name for non-PE emails

Integration Pattern

The recommended integration model is **pull**: the downstream system (e.g., IDP) reads from the `triage-complete` queue on your primary Service Bus namespace using shared-access or RBAC credentials. This avoids cross-namespace authentication complexity.

If push to an external namespace is needed, set `TRIAGE_COMPLETE_SB_NAMESPACE` to the target FQDN. The agent will authenticate via `DefaultAzureCredential` and includes dead-letter fallback if the external send fails.

Cosmos DB Fields

When pipeline mode is active, each processed email record in Cosmos DB includes:

Field	Example (`full`)	Example (`triage-only`)
`pipelineMode`	`"full"`	`"triage-only"`
`stepsExecuted`	`["relevance","classification","extraction"]`	`["relevance"]`

Dashboard Indicators

- A **pipeline mode badge** is displayed in the dashboard header ("Full Pipeline" or "Triage Only")
- Emails processed in triage-only mode show a "Skipped (triage-only)" label in the status column
- The `triage-complete` queue appears in the queue monitor when using the primary namespace

Authentication (Entra ID Easy Auth)

The web dashboard is protected by **Azure App Service Easy Auth** (also known as built-in authentication) using **Microsoft Entra ID** (formerly Azure AD). This means authentication is handled at the platform level — the application code itself does not implement any login logic.

How It Works

```
┌──────────────┐      ┌──────────────────────────┐      ┌──────────────────┐
│   Browser    │──────▶│  App Service Easy Auth   │──────▶│  FastAPI App     │
│              │      │  (authentication layer)  │      │  (dashboard)     │
│              │◀──────│                          │◀──────│                  │
└──────────────┘      └──────────────────────────┘      └──────────────────┘
                                  │
                                  ▼
                      ┌──────────────────┐
                      │  Microsoft       │
                      │  Entra ID        │
                      │  (login.ms.com)  │
                      └──────────────────┘
```

1. A user navigates to the App Service URL.
2. App Service intercepts the request **before** it reaches the FastAPI app.
3. If the user is not authenticated, they are redirected to the Microsoft Entra ID login page.
4. After successful login, Entra ID issues a token and redirects back to the App Service.
5. App Service validates the token and forwards the request to the FastAPI app with authentication headers.
6. The app never handles credentials — it only sees pre-authenticated requests.

Key Configuration

Authentication is configured via `authsettingsV2` on the App Service resource:

Setting	Value	Purpose
`platform.enabled`	`true`	Enables the authentication layer
`requireAuthentication`	`true`	All requests must be authenticated
`unauthenticatedClientAction`	`RedirectToLoginPage`	Unauthenticated users are redirected to login
`redirectToProvider`	`azureActiveDirectory`	Uses Entra ID as the identity provider
`openIdIssuer`	`https://login.microsoftonline.com/{tenantId}/v2.0`	Tenant-specific token issuer
`clientId`	App Registration client ID	Identifies the app to Entra ID
`allowedAudiences`	`[clientId]`	Validates token audience (must be the client ID without `api://` prefix)
`tokenStore.enabled`	`true`	Stores session tokens server-side

Entra ID App Registration

Easy Auth requires an **App Registration** in Microsoft Entra ID. This is a one-time setup:

1. **Create the App Registration** (Azure Portal → Entra ID → App registrations → New registration):
    - **Name**: e.g., `app-docproc-dev-dashboard`
    - **Supported account types**: Single tenant (this organization only)

- **Redirect URI**: `https://<your-app-name>.azurewebsites.net/.auth/login/aad/callback` (type: Web)

2. **Enable ID tokens**: Go to Authentication → check "ID tokens" under Implicit grant
3. **Create a Service Principal** (if not auto-created):

```
az ad sp create --id <appId>
```

4. **Note the Application (client) ID** — this is used as both `clientId` and `allowedAudiences` in the auth config.

Infrastructure as Code

The authentication configuration is defined in Bicep at infrastructure/modules/web-app.bicep:

```
resource authSettings 'Microsoft.Web/sites/config@2023-12-01' = {
  parent: webApp
  name: 'authsettingsV2'
  properties: {
    platform: { enabled: true }
    globalValidation: {
      requireAuthentication: true
      unauthenticatedClientAction: 'RedirectToLoginPage'
      redirectToProvider: 'azureActiveDirectory'
    }
    identityProviders: {
      azureActiveDirectory: {
        enabled: true
        registration: {
          openIdIssuer:
'${environment().authentication.loginEndpoint}${authTenantId}/v2.0'
          clientId: authClientId
        }
        validation: {
          allowedAudiences: [ authClientId ]
        }
      }
    }
    login: { tokenStore: { enabled: true } }
  }
}
```

The `authClientId` and `authTenantId` are passed as parameters from infrastructure/main.bicep.

Common Auth Pitfalls

Issue	Symptom	Fix
Missing Service Principal	Login page shows `AADSTS700016` error	Run `az ad sp create --id <appId>`
ID tokens not enabled	Login redirects but fails	Enable "ID tokens" in App Registration → Authentication

Issue	Symptom	Fix
Wrong audience format	`401 Unauthorized` after login	Use the client ID directly (e.g., `9a517e48-...`), NOT `api://9a517e48-...`
Multiple identity providers	Unexpected redirect or `500` errors	Remove all providers except `azureActiveDirectory` from `authsettingsV2`
Startup command not set	Container crashes with exit code 3	Set via ARM REST API (see Deployment section)

Auth Endpoints

Once Easy Auth is enabled, these endpoints are available automatically:

Endpoint	Purpose
`/.auth/login/aad`	Initiates Entra ID login
`/.auth/login/aad/callback`	OAuth callback (redirect URI)
`/.auth/me`	Returns the authenticated user's claims (JSON)
`/.auth/logout`	Signs out the user

Deploying to Azure

This section provides step-by-step instructions for deploying the application to Azure.

Prerequisites

- **Azure CLI** installed and authenticated (`az login`)
- **Azure CLI Logic App extension**: `az extension add --name logic`
- An **Azure subscription** with permissions to create resources
- An **Entra ID App Registration** for the web dashboard (see Authentication section)
- An **Entra ID App Registration** for the Graph API / SharePoint service principal

Step 1: Provision Infrastructure

Deploy all Azure resources using the provided Bicep templates:

```
# Login and set subscription
az login
az account set --subscription "<subscription-id>"

# Create resource group
az group create --name rg-docproc-dev --location westeurope

# Deploy infrastructure (creates all resources + role assignments)
az deployment group create \
  --resource-group rg-docproc-dev \
  --template-file infrastructure/main.bicep \
  --parameters infrastructure/parameters/dev.bicepparam
```

This creates: App Service Plan, Web App, Cosmos DB, Service Bus, Storage Account, Document Intelligence, Log Analytics, Key Vault, Logic Apps (email + SFTP), and all RBAC role assignments.

Step 2: Pre-Provision Key Vault Secrets

The Key Vault is created by Bicep but **secrets must be manually provisioned** before deploying the Logic Apps and webapp. These secrets contain credentials for external service principals that cannot use managed identity.

Secret Name	Purpose	Where Used
`sharepoint-client-secret`	Entra ID client secret for SharePoint Graph API uploads	SFTP Logic App (`Upload_to_SharePoint` action)
`graph-client-secret`	Entra ID client secret for Microsoft Graph API (email attachments)	Web App (`graph_tools.py`)
`sftp-private-key`	SSH private key for SFTP connector authentication	SFTP-SSH API Connection (Bicep `getSecret()`)

Create the secrets:

```
$keyVaultName = "<your-key-vault-name>"   # e.g., kv-docproc-dev-izr2ch55

# SharePoint client secret (from Entra ID app registration)
az keyvault secret set `
  --vault-name $keyVaultName `
  --name "sharepoint-client-secret" `
  --value "<your-sharepoint-client-secret>"

# Graph API client secret (from Entra ID app registration)
az keyvault secret set `
  --vault-name $keyVaultName `
  --name "graph-client-secret" `
  --value "<your-graph-api-client-secret>"

# SFTP private key (from file)
az keyvault secret set `
  --vault-name $keyVaultName `
  --name "sftp-private-key" `
  --file "<path-to-your-sftp-private-key.pem>"
```

```
# Bash equivalent
KEY_VAULT_NAME="<your-key-vault-name>"

az keyvault secret set \
  --vault-name "$KEY_VAULT_NAME" \
  --name "sharepoint-client-secret" \
  --value "<your-sharepoint-client-secret>"

az keyvault secret set \
  --vault-name "$KEY_VAULT_NAME" \
  --name "graph-client-secret" \
  --value "<your-graph-api-client-secret>"
```

```
az keyvault secret set \
  --vault-name "$KEY_VAULT_NAME" \
  --name "sftp-private-key" \
  --file "<path-to-your-sftp-private-key.pem>"
```

> **Note**: The `sftp-private-key` uses `--file` (not `--value`) because PEM keys are multi-line. The other two secrets are single-line client secret values from Entra ID App Registrations.

Step 3: Deploy with the Deployment Script

The provided `deploy_updates.ps1` script deploys all components in the correct order:

```
.\deploy_updates.ps1
```

What the script does:

Step	Component	Action
1a	Email Logic App	Deploys the workflow definition from `logic-apps/email-ingestion/workflow.json`
1b	SFTP Logic App	Fetches `sharepoint-client-secret` from Key Vault, reads current Logic App parameters (preserving `$connections`), injects the secret, and deploys the workflow definition with parameters
2	Web App	Zips `src/`, `utils/`, `requirements.txt`, `startup.sh` and deploys via `az webapp deploy`
3	App Settings	Ensures `STORAGE_ACCOUNT_URL`, `KEY_VAULT_URL`, `GRAPH_CLIENT_ID`, `GRAPH_TENANT_ID` are set on the Web App
4	RBAC	Assigns `Key Vault Secrets User` role to the Web App managed identity on the Key Vault

> **Customizing for your environment**: Edit the variables at the top of the script (`$resourceGroup`, `$logicAppName`, `$sftpLogicAppName`, `$webAppName`, `$keyVaultName`) and the `GRAPH_CLIENT_ID` / `GRAPH_TENANT_ID` values in the app settings section.

Step 4: Add Remaining App Settings

After infrastructure provisioning, add the app settings that map to the variable names the application code expects:

```
APP_NAME="<your-app-name>"   # e.g., app-docproc-dev-izr2ch55woa3c
RG="rg-docproc-dev"

az webapp config appsettings set \
  --resource-group $RG --name $APP_NAME \
  --settings \
    COSMOS_ENDPOINT="https://<cosmos-account>.documents.azure.com:443/" \
    SERVICEBUS_NAMESPACE="<servicebus-name>" \
    PIPELINE_MODE="triage-only" \
    PYTHONPATH="/home/site/wwwroot"
```

> **Note**: The Bicep templates create `COSMOS_DB_ENDPOINT` and `SERVICE_BUS_NAMESPACE`, but the application code expects `COSMOS_ENDPOINT` and `SERVICEBUS_NAMESPACE`. Add both naming variants to avoid `KeyError` crashes.

Step 5: Set the Startup Command

The startup command **must be set via the ARM REST API** because `az webapp config set --startup-file` silently fails for Python Linux apps:

```
APP_NAME="<your-app-name>"
RG="rg-docproc-dev"
SUB_ID="<subscription-id>"

az rest --method PATCH \
  --url
"https://management.azure.com/subscriptions/$SUB_ID/resourceGroups/$RG/providers/Microso
ft.Web/sites/$APP_NAME?api-version=2023-12-01" \
  --body '{"properties":{"siteConfig":{"appCommandLine":"gunicorn src.webapp.main:app --
workers 2 --worker-class uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000"}}}'
```

> **Why ARM REST API?** Azure CLI's `--startup-file` flag for Python apps has a known issue where it appears to succeed but doesn't persist the value. Using the ARM REST API with `appCommandLine` is reliable.

Step 6: Verify Deployment

```
# Check the app is running
az webapp show --resource-group rg-docproc-dev --name <your-app-name> \
  --query "{state:state, url:defaultHostName}" -o table

# Check for startup errors in the logs
az webapp log tail --resource-group rg-docproc-dev --name <your-app-name> --timeout 30

# Test the endpoint (should return 302 redirect to login, or 401 if Easy Auth is
enabled)
curl -s -o /dev/null -w "HTTP_STATUS=%{http_code}" "https://<your-app-
name>.azurewebsites.net/"
```

A `302` (redirect to login) or `401` response confirms the app is running and Easy Auth is active. Open the URL in a browser to sign in with your Entra ID credentials and access the dashboard.

Step 7: Subsequent Deployments

For code-only updates (no infrastructure changes), run the deploy script:

```
.\deploy_updates.ps1
```

Or manually deploy just the webapp:

```
# Quick redeploy (PowerShell)
Compress-Archive -Path src, utils, requirements.txt, startup.sh -DestinationPath
deploy.zip -Force
az webapp deploy --resource-group rg-docproc-dev --name <your-app-name> --src-path
deploy.zip --type zip --async true
Remove-Item deploy.zip
```

> **Important**: The App Service uses Oryx build system with `SCM_DO_BUILD_DURING_DEPLOYMENT=true`. This means `pip install -r requirements.txt` runs automatically during deployment. The built app is served from `/tmp/<hash>/`, not `/home/site/wwwroot/` — do NOT use `--chdir` in the startup command.

Deployment Checklist

Step	Action	Verification
1	Provision infrastructure with Bicep	`az deployment group show` returns `Succeeded`
2	Pre-provision Key Vault secrets	`az keyvault secret list --vault-name <kv>` shows 3 secrets
3	Create Entra ID App Registrations	Apps appear in Azure Portal → Entra ID → App registrations
4	Run `.\deploy_updates.ps1`	All steps report success (green output)
5	Add remaining app settings	`az webapp config appsettings list` shows all settings
6	Set startup command via ARM REST API	`az webapp show --query siteConfig.appCommandLine` returns the gunicorn command
7	Verify app is running	Browser shows Entra ID login → dashboard after sign-in
8	Verify SFTP Logic App	Upload a test file to SFTP `/in/` → check Logic App run history

Connections and Secrets Management

This section explains how secrets, API connections, and authentication are managed across the solution.

Key Vault Architecture

Azure Key Vault is the centralized secret store for all sensitive credentials. Secrets are **pre-provisioned manually** (not created by Bicep) and consumed at deploy time or runtime.

```
┌──────────────────────────────────────────────────────────┐
│                    Azure Key Vault                       │
│                                                          │
│  ┌──────────────────────────┐                            │
│  │ sharepoint-client-secret │──▶ SFTP Logic App          │
│  │                          │   (injected at deploy time │
│  │                          │    via deploy_updates.ps1) │
│  └──────────────────────────┘                            │
│                                                          │
│  ┌──────────────────────────┐                            │
│  │ graph-client-secret      │──▶ Web App                 │
```

```
│  │                          │   (fetched at runtime via  │
│  │                          │    SecretClient SDK)       │
│  └──────────────────────────┘                            │
│                                                          │
│  ┌──────────────────────────┐                            │
│  │ sftp-private-key         │──▶ SFTP-SSH API Connection │
│  │                          │   (injected at Bicep       │
│  │                          │    deploy via getSecret())  │
│  └──────────────────────────┘                            │
└──────────────────────────────────────────────────────────┘
```

How Secrets Are Consumed

Secret	Consumer	Injection Method	When
`sharepoint-client-secret`	SFTP Logic App (`Upload_to_SharePoint`)	`deploy_updates.ps1` fetches from KV and injects via `--parameters`	Deploy time
`graph-client-secret`	Web App (`graph_tools.py`)	`SecretClient` SDK with managed identity	Runtime
`sftp-private-key`	SFTP-SSH API Connection resource	Bicep `getSecret()` function	Infrastructure deploy

RBAC Requirements for Key Vault

The Key Vault uses **RBAC authorization** (not access policies). The following identities need roles:

Identity	Role	Purpose
Web App managed identity	`Key Vault Secrets User`	Read `graph-client-secret` at runtime
Deployer (your Azure CLI identity)	`Key Vault Secrets User`	Read secrets during `deploy_updates.ps1` execution
Deployer (your Azure CLI identity)	`Key Vault Secrets Officer`	Create/update secrets (one-time provisioning)

> The `deploy_updates.ps1` script automatically assigns `Key Vault Secrets User` to the Web App managed identity. Deployer roles must be assigned manually or via Bicep.

API Connections (Logic App Managed Connectors)

Logic Apps use **managed connectors** that are provisioned as `Microsoft.Web/connections` resources. Each connection authenticates differently:

Connection	Auth Type	How Credentials Are Managed
`sftpwithssh` (trigger)	SSH private key	Key from Key Vault via Bicep `getSecret()` at infra deploy
`sftpwithssh-1` (actions)	SSH private key	Same key, same injection method
`documentdb` (Cosmos DB)	Managed identity	No secrets — uses Logic App system-assigned MI
`azureblob` (Storage)	Managed identity	No secrets — uses Logic App system-assigned MI

Connection	Auth Type	How Credentials Are Managed
`servicebus` (Service Bus)	Managed identity	No secrets — uses Logic App system-assigned MI

Graph API Authentication

The solution uses Microsoft Graph API in two contexts with different auth patterns:

1. SFTP Logic App → SharePoint (File Uploads)

- Uses an HTTP action with `ActiveDirectoryOAuth` client credentials
- The `sharepointClientSecret` is a Logic App workflow parameter injected at deploy time from Key Vault
- This is necessary because the Logic App Consumption tier does not support managed identity for service principal-based Graph API calls

2. Web App → Graph API (Email Attachments)

- The `GraphAPITools` class in `src/agents/tools/graph_tools.py` uses `ClientSecretCredential`
- The client secret is fetched from Key Vault at runtime using the Web App's managed identity
- Falls back to `DefaultAzureCredential` if no client credentials are available (but this only works for the app's own identity, not for accessing other users' mailboxes)
- Required env vars on the Web App: `GRAPH_CLIENT_ID`, `GRAPH_TENANT_ID`, `KEY_VAULT_URL`

Entra ID App Registrations

Two separate app registrations are used:

App Registration	Purpose	API Permissions	Secret Location
Graph API / SharePoint SP	Email attachment download + SharePoint file upload	`Mail.Read` (Application), `Sites.ReadWrite.All` or `Sites.Selected` (Application)	Key Vault: `graph-client-secret` and `sharepoint-client-secret`
Dashboard (Easy Auth)	Web dashboard authentication	N/A (used for sign-in only)	No secret needed (platform-managed)

> If the Graph API and SharePoint app registrations use the **same** Entra ID app, then `graph-client-secret` and `sharepoint-client-secret` will have the same value. You can still store them as separate secrets for clarity and independent rotation.

Setting Up Connections for a New Tenant

When deploying to a new tenant, follow this sequence:

1. Create Entra ID App Registrations:

```
# Graph API / SharePoint service principal
az ad app create --display-name "docproc-graph-sp" \
  --required-resource-accesses '[
    {"resourceAppId": "00000003-0000-0000-c000-000000000000", "resourceAccess": [
      {"id": "810c84a8-4a9e-49e6-bf7d-12d183f40d01", "type": "Role"},
      {"id": "9492366f-7969-46a4-8d15-ed1a20078fff", "type": "Role"}
    ]}
```

```
  ]'

# Grant admin consent (requires Global Admin or Privileged Role Administrator)
az ad app permission admin-consent --id <app-id>

# Create a client secret (valid for 2 years)
az ad app credential reset --id <app-id> --years 2
```

2. Provision Key Vault secrets (see Step 2 above)

3. Deploy infrastructure with your tenant-specific values in `parameters/dev.bicepparam`

4. Run the deployment script `.\deploy_updates.ps1`

Troubleshooting

Common Issues

1. Authentication Errors

```
Azure authentication failed
```

Solution:

```
az login
az account show  # Verify correct subscription
```

2. Missing Environment Variables

```
KeyError: 'SERVICEBUS_NAMESPACE'
```

Solution: Ensure your `.env` file exists and contains all required variables.

3. Service Bus Connection Issues

```
Unable to connect to Service Bus
```

Solution: Verify your Azure identity has `Azure Service Bus Data Sender/Receiver` role.

4. Cosmos DB Access Denied

```
Authorization token not valid
```

Solution: Assign `Cosmos DB Built-in Data Contributor` role to your identity.

Diagnostic Commands

```
# Check environment configuration
python utils/diagnose.py

# Test Azure connectivity
python utils/test_connectivity.py

# View queue contents
python src/peek_queue.py
```

Logs

The agent logs to stdout. For detailed logging:

```
python src/agents/run_agent.py 2>&1 | tee agent.log
```

Contributing

1. Create a feature branch
2. Make your changes
3. Run tests to ensure nothing is broken
4. Submit a pull request

License

Copyright © 2025. All rights reserved.